Part No .1

.NET SCENARIO BASED QUESTIONS AND ANSWERS.

Q: Your API is running slow when handling heavy data queries. How would you diagnose and fix it?

Answer:

First, I would use logging and profiling tools like **Application Insights**, **MiniProfiler**, or **Serilog** to trace the requests and identify bottlenecks. Then, I would inspect the **SQL query execution plan** (and if using Entity Framework, I'd review the generated SQL using `.ToQueryString()`) to look for optimization opportunities.

After that, I would:

- Use **`AsNoTracking()`** for read-only data to improve EF Core performance.
- Implement **paging** using methods like `Skip()` and `Take()` to reduce data load.
- Check for appropriate **indexes** in the database to speed up query execution.
- Use **caching mechanisms** (like `MemoryCache` or `Redis`) for frequently accessed data to reduce load on the database.

Q: How do you implement consistent exception handling in a large enterprise application?

Answer:

I use **middleware at the global level** in **ASP.NET Core** for centralized error handling. In this middleware, I catch all exceptions and return a proper response in a **custom error format** along with appropriate **HTTP status codes**.

- To handle specific exceptions, I use **try-catch blocks** in the **service layer**.
- Every exception is logged using **logging tools** like **Serilog**, **NLog**, or **Application Insights**.
- The error response is standardized, for example:
  `{ statusCode: 500, message: "Internal Server Error" }`.

Aaqib Khan Niazi

Q: If the client needs real-time notifications whenever a record is updated, what architecture would you use?

Answer:

I would use **SignalR**, which is ASP.NET Core's real-time communication library. When data is updated on the backend, an update is pushed to connected clients through the **SignalR hub**.

The architecture would look like this:

- **Client-side**: A JavaScript/TypeScript client connects to SignalR.
- **Server-side**: I would create a **Hub** from which I call `Clients.All.SendAsync("RecordUpdated")` to notify all connected clients.
- For **security**, I would implement **JWT token-based authentication**.

Q: You have a SaaS app with multiple clients (tenants). How would you ensure data isolation?

Answer:

1. To ensure tenant-specific data isolation, I would consider the following three options:
2. **Database per tenant** – Each tenant has a separate database. This offers the highest level of security and isolation.
3. **Schema per tenant** – A single database, but each tenant has its own schema.
4. **Shared schema with TenantID** – All tenants share the same tables, but their data is separated using a **TenantID** column.

At the code level:

- I would create a **TenantResolver** in the **middleware** to identify the tenant from the request headers, domain, or JWT.
- In **EF Core**, I would apply **query filters** to ensure that only the data belonging to the specific tenant is fetched.

Aaqib Khan Niazi

Q: If your web app has a task that takes 5–10 minutes (e.g., report generation), what would you do to ensure the UI doesn't block but the task still gets executed?

Answer:

In this case, I would use **background processing**.

- For **ASP.NET Core**, I use `IHostedService` or `BackgroundService` for background processing.
- For more complex or delayed tasks, I use libraries like **Hangfire** or **Quartz.NET**.
- The frontend receives an immediate response such as:
  "Report is being generated."
- Once the task is complete, I notify the client using **SignalR** or implement **polling**.

Q: You have multiple clients using your API. Now you need to make breaking changes in the API. What approach would you take?

Answer:

I would use **Web API versioning**.

- In **ASP.NET Core**, I use the **Microsoft.AspNetCore.Mvc.Versioning** package:
  1. **URL-based:** `/api/v1/products`
  2. **Header-based:** `X-Version: 2`
  3. **Query-string based:** `/api/products?api-version=1.0`
- I create separate controllers for each version (e.g., `ProductsV1Controller`, `ProductsV2Controller`).
- This way, I can introduce new features without affecting old clients.

Q: If multiple users are updating the same record, how would you handle it to prevent data overwriting?

Answer:

Aaqib Khan Niazi

I would use **optimistic concurrency** in EF Core:

- I add a **RowVersion** or **Timestamp** property to the model.
- When an update occurs, EF includes the **RowVersion** in the SQL `WHERE` clause.
- If someone else has already updated the record, the update will fail.
- In that case, I show the user a message:
  "Someone else has modified this record. Please reload."

Q: A file is being uploaded from the client. How would you ensure security is maintained and no malicious files are accepted?

Answer:

- I would allow **only specific file types** (e.g., PDF, DOCX) by validating both the **file extension** and **MIME type**.
- I would enforce a **maximum file size limit** (e.g., 5 MB).
- I would **rename the file** before saving it on the server — avoiding saving it directly with the original name.
- I would use an **antivirus API** (such as ClamAV) to scan the file.
- I would make sure the **upload directory is not web-accessible**, so only authorized users can download the files.

Q: Your app has gone viral and now you need to handle high traffic. What would your plan be?

Answer:

1. I would use a **load balancer** (e.g., Azure Application Gateway, Nginx).
2. I would **scale out** the app by running it on multiple instances.
3. To keep sessions server-independent, I would use a **distributed cache** like Redis.
4. I would host static content (images, JS) on a **CDN**.
5. I would optimize the database by:
   a. Read replicas
   b. Indexes
   c. Caching

Aaqib Khan Niazi

Q: Your system uses JWT authentication, and the token expires in 30 minutes. How would you prevent users from having to log in repeatedly?

Answer:

I would implement a **refresh token mechanism** along with JWT:

1. When the user logs in, a **short-lived access token (JWT)** and a **long-lived refresh token** are generated.
2. I securely store the refresh token in the database or cache.
3. When the JWT expires, the client uses the refresh token to obtain a new JWT.
4. To protect against refresh token hijacking, I also store the **IP address** and **device fingerprint**.
5. I implement a **revoke mechanism** to invalidate refresh tokens when necessary.

Q: You need to follow Clean Architecture in your project. How would you design the structure?

Answer:

1. I divide the system into separate layers:
2. **Domain Layer** – Entities, Value Objects, Interfaces (pure business logic)
3. **Application Layer** – Use Cases, DTOs, Interfaces, Services (no EF or infrastructure logic)
4. **Infrastructure Layer** – EF Core, External APIs, File System access
5. **API Layer** – Controllers, Dependency Injection, Filters, Authentication

Aaqib Khan Niazi

- **Q: Your API is public, and some clients are misusing it (e.g., 1000 requests per minute). How would you control this?**
- **Answer:**
- I would use **Rate Limiting Middleware**:

- In ASP.NET Core, I use the **AspNetCoreRateLimit** package.
- Define rules per IP/client, for example:
  100 requests per 5 minutes.
- In a distributed environment, the rate limit should be maintained using **Redis**.
- When the limit is exceeded, I return a **429 (Too Many Requests)** status code.

- **Advanced:**
- I use the **token bucket** or **sliding window** algorithm for better rate control.
- By implementing an **API key system**, I can define different plans (Free, Premium, etc.).

Q: You have multiple microservices. If an error occurs, how would you trace its origin?

Answer:

I implement **centralized logging** using:

- **Serilog** or **NLog** with **Seq** or the **ELK Stack** (Elasticsearch + Kibana).

  I include a **CorrelationId** in every log:

- A unique ID is generated from the start of the request until the response.
- This ID is propagated across all services via headers.

Q: You need to delete a record but not physically — just mark it as deleted. What approach would you take?

Answer:

I would implement **Soft Delete**:

- Add an **IsDeleted** or **DeletedAt** field to the entity.
- Use **EF Core Global Query Filters**, for example:

```
modelBuilder.Entity<User>().HasQueryFilter(x => !x.IsDeleted);
```

Aaqib Khan Niazi

- JWhen deleting, I update the record instead of removing it.

Benefits:

- Maintains an audit trail.
- Data can be recovered in the future.

Q: Your Web API is exposed. If a user sends extra fields, those fields are also getting updated. How would you prevent this?

Answer:

- I use **DTOs** — never bind entities directly.
- I allow only specific fields through **AutoMapper** or manual mapping.
- Using attributes like **[Bind]**, **[FromBody]**, and **[FromForm]** ensures controlled binding.

Aaqib Khan Niazi

Q: If two services depend on each other (A → B and B → A), you get a circular dependency error at runtime. What would you do?

Answer:

This is a design flaw. To fix it:

1. Break the dependencies using **interface abstraction**.
2. Use the **Mediator pattern** (like MediatR) or **events** to achieve loose coupling.
3. If necessary, use a **factory** or **service locator** pattern:
    a. Instead of injecting a service directly via the constructor, resolve it at runtime using **IServiceProvider**.

Q: In your app, you need to maintain an audit log for every update/create/delete action — including user, timestamp, and data changes. How would you do it?

Answer:

I use two approaches for audit trails:

1. EF Core Change Tracker:

- Override `DbContext.SaveChanges()`, check all entities with changes.
- Insert log entries into a separate **AuditLogs** table containing:
  Table name, operation type, userId, old values, new values, timestamp.

2. Interceptor or Middleware Level:

- Use domain events or action filters to trace requests and log changes.
- Get the current user from `HttpContext.User.Identity.Name`.
- Best practice is to perform **async logging** to separate storage (e.g., RabbitMQ → Database) for better performance.

Q: You are deploying an app to production and need to update the database schema. Live users are connected. How would you perform the migration?

Answer:

Database migrations in production are risky. I follow these steps:

1. Use **EF Core Migrations**, but carefully review the generated SQL scripts.
2. First, test the migration in a **staging environment**.
3. Follow a **zero-downtime strategy**:
   a. First, add new columns as **nullable or optional**.
   b. Then update the app to use the new schema.
   c. Later, drop the old columns.
4. Use **transactional scripts** or migration pipelines (e.g., Azure DevOps, Octopus).
5. Always take a **backup** before starting.

Q: Your API is used by both admin and normal users. How would you control access for each route?

Answer:

Aaqib Khan Niazi

I implement **Role-Based Authorization**:

1. Include user roles in the JWT token.
2. Apply `[Authorize(Roles = "Admin")]` attribute on controllers or actions.
3. For complex policies:
    a. Create an `IAuthorizationHandler` with custom logic (e.g., allow only the owner or admin).
4. I also configure **Swagger** to hide unauthorized routes.

Q: You have cached data (e.g., product list). Now the backend data is updated. How do you ensure the client gets the updated data?

Answer:

This is a cache invalidation problem. Solutions include:

1. **Time-based expiration** (e.g., 5 minutes):
   Simple but risks serving stale data.
2. **Manual invalidation:**
   When a product is updated, I delete the related cache key.
3. **Publish/Subscribe model:**
   Use Redis pub/sub or a message queue.
   On update, a notification is sent and the cache is refreshed.
4. Use the **Cache Aside pattern:**
   Always try to get data from the cache; if it's not present, load it from the database and then set it in the cache.

Q: You are building an API that allows downloading large files. How would you ensure that if the download is interrupted, it can be resumed?

Answer:

This feature is called **Range Requests**. The approach is:

1. The API sends the header `Accept-Ranges: bytes` in the response.
2. If the client wants to resume, it sends a `Range` header, for example:
   `Range: bytes=1000-`
3. The server starts streaming the file from that offset and responds with status **206 Partial Content**.

Aaqib Khan Niazi

4. In ASP.NET Core, this requires using `FileStreamResult` along with custom logic to handle the range requests.

Q: The client is submitting a heavy form (e.g., image upload + metadata). Sometimes the request times out. How would you handle this?

Answer:

To fix timeouts, I check multiple levels:

Frontend:

1. Set client timeout (in JS, Axios config) to more than 30 seconds.

Backend:

2. Increase Kestrel server timeouts (e.g., `RequestTimeout`, `MaxRequestBodySize`).

IIS / Nginx / Reverse proxy:

3. Ensure their timeouts are not overriding the backend settings.

Solution:

4. Offload long operations to a background queue (e.g., Hangfire).
5. Provide the frontend with an async ticket or response like:
   "Processing started."

Q: You are building a scalable app where multiple entities are updated within a single transaction. How would you handle this?

Answer:

- I use the **Repository** and **Unit of Work** patterns:
- A separate **Repository** for each entity handling read/write logic.
- A common **IUnitOfWork** interface that:

Code:
```
public interface IUnitOfWork {

    Task<int> SaveChangesAsync();
```

Aaqib Khan Niazi

```
    IUserRepository Users { get; }

    IOrderRepository Orders { get; }

}
```

Q: In your async method, the UI thread or request is hanging. How would you diagnose this?

Answer:

A deadlock usually occurs when an async method is called synchronously using `.Result` or `.Wait()` within a synchronization context (e.g., a web request).

Solution:

- Always use `await`; avoid `.Result`.
- Keep the entire async method chain fully asynchronous.
- In ASP.NET Core, there is no SynchronizationContext, but in legacy ASP.NET or WinForms, use `ConfigureAwait(false)` to avoid capturing the UI thread.

Q: Your project is using C# 8+ with nullable reference types enabled. How would you use them and what are the benefits?

Answer:

Enabling nullable reference types (e.g., `string?`) allows the compiler to perform static analysis:

- It warns you if there is a chance of null dereference.
- This helps reduce runtime null reference exceptions.

Best practices:

- Use null checks (e.g., `if (x != null)`).
- Use the null-forgiving operator (`!`) carefully.
- Practice defensive programming — make null checks mandatory in public APIs.

Aaqib Khan Niazi

Q: If your web server is shutting down (e.g., app stop or deployment), how would you ensure a graceful shutdown of background jobs?

Answer:

In ASP.NET Core, I use **IHostApplicationLifetime** or **IApplicationLifetime**:

- Subscribe to the **ApplicationStopping** event and send a cancellation token to background jobs.
- Handle the cancellation token properly in the `ExecuteAsync` method of the background service.
- Ensure the job exits cleanly without data corruption or half-processed states.

Q: You have a SaaS app with Basic and Premium users. Premium users are allowed more API calls. How would you implement this?

Answer:

- I implement rate limiting at the middleware or filter level based on the user's plan.
- User plan details are stored in the JWT token or database.
- I maintain a per-user request counter in Redis or in-memory store.
- I define custom policies such as:

  Basic: 60 requests per minute
  Premium: 500 requests per minute

Q: You need to insert/update 10,000 records in the database without degrading performance. What would you do?

`Answer:`

Bulk operations are slow by default in EF Core.

Options:

- Use third-party libraries like **EFCore.BulkExtensions** or **Dapper**.
- For SQL Server, use the **SqlBulkCopy** class.
- EF batching optimizations:
  - Set AutoDetectChangesEnabled = false.
  - Call SaveChanges() in batches (e.g., every 1000 records).

Aaqib Khan Niazi

- Avoid calling SaveChanges() inside a loop.

Q: You are exposing multiple endpoints. Each endpoint should have a different rate limit. How would you configure this?

Answer:

- I define per-endpoint rules in the AspNetCoreRateLimit configuration file.

Code:

```
{
  "Endpoint": "*:/api/login",
  "Limit": 5,
  "Period": "1m"
}
```

- I apply rules based on route name or path when writing custom middleware.
- Advanced approach: I create attribute-based throttling like `[RateLimit(5, "1m")]`.

Q: In your microservice, both database update and event publishing are happening. How do you ensure both remain consistent?

Answer:

I use the **Outbox Pattern**:

- Within a transaction, both the database update and insertion of the event into an "outbox" table happen.
- A background worker (Outbox Processor) publishes the event from the outbox table to the event queue (e.g., RabbitMQ).
- This provides eventual consistency without using distributed transactions.

Aaqib Khan Niazi

Q: Your app has two separate databases (e.g., identity and business data), both using EF Core DbContexts. How would you manage them?

Answer:

- Create a separate `DbContext` class for each database.
- In Startup, register both with their respective options separately.

Code:
```
services.AddDbContext<AppDbContext>(...);

services.AddDbContext<AuthDbContext>(...);
```

- I use a scoped lifetime.
- If a cross-context transaction is needed, I manually begin it using `IDbContextTransaction`.

Q: If you are using raw SQL in EF Core, how do you prevent SQL injection?

Answer:

I use EF Core `FromSqlInterpolated` or `ExecuteSqlInterpolated` methods.

Code:
```
context.Users

.FromSqlInterpolated($"SELECT * FROM Users WHERE Name = {userName}")
```

- It internally generates a parameterized query.
- Never use string concatenation in SQL statements.

Q: In your app, emails/SMS need to be sent after user actions. Sending them in real time causes delays. What's the solution?

Answer:

I create a **queued background job system**:

1. The job is placed in a **queue** (either in-memory or a message queue).

Aaqib Khan Niazi

2. A **background worker** (e.g., `BackgroundService`) processes the job asynchronously.

Tools:

- **Hangfire** (simple dashboard + retry support)
- **Azure Queue** / **Service Bus**
- **RabbitMQ** with **MassTransit**

Q: Your app makes calls to a third-party API, but it occasionally fails (e.g., timeout). How would you make it more reliable?

Answer:

I use the **Polly** library in .NET for:

Retry with exponential backoff:

Code:
Policy

```
  .Handle<HttpRequestException>()

  .WaitAndRetryAsync(3, retryAttempt => TimeSpan.FromSeconds(Math.Pow(2,
retryAttempt)));
```

Circuit Breaker:

- If 5 out of 10 calls fail, the circuit breaker should open for 30 seconds.

Q: An error occurs in production. How would you trace which request it was and which user it belonged to?

Answer:

At the start of each request, I generate a **Correlation ID** (GUID).

This ID is:

- Attached to all logs

Aaqib Khan Niazi

- Included in the response headers
- Present in log messages across every layer of the application

Tools: Serilog + Enrichers + Middleware

Q: You have a custom middleware in your ASP.NET Core app, but it's not working. What would you check?

Answer:

- In ASP.NET Core, the **execution order of middleware matters** — the pipeline runs **top to bottom**.
- If `UseAuthentication()` or `UseAuthorization()` is placed too low, it won't work properly.
- If my custom middleware is **short-circuiting** the request (e.g., using `return;`), then the next middleware in the pipeline won't execute.

Solution: Carefully maintain the **middleware order** in `Startup.cs` or `Program.cs`.

Q: Your app is used worldwide. You need to show each user their local time, but the backend stores everything in UTC. How would you handle this?

Answer:

1. I always store **UTC** in the backend (both in the database and APIs).
2. The frontend sends the user's **timezone or offset** (e.g., using JavaScript's `Intl.DateTimeFormat().resolvedOptions().timeZone`).
3. I apply the conversion logic either on the **server** or **client** side:
    a. In C#, use `.ToLocalTime()` for conversion.

Aaqib Khan Niazi

Q: You have a complex config file (appsettings.json). You don't want to access it everywhere using dynamic strings. What's the best practice?

Answer:

I map the configuration to **strongly typed POCO classes**.

services.Configure<MySettings>(Configuration.GetSection("MySettings"));

In Controller or Service:
public MyController(IOptions<MySettings> settings)

Q: Your LINQ query is causing performance issues. How would you diagnose and fix it?

Answer:

I inspect the query using `.ToQueryString()` or a profiler.

Common problems:

- Including too many navigations (`.Include().ThenInclude()`)
- Missing indexes
- Using `SELECT *` instead of projecting only needed fields
- Calling `.ToList()` too early (early materialization)

Fixes:

- Add proper indexing in the database
- Use `Select` to limit retrieved columns
- Avoid the **N+1 query problem**

Q: A user is uploading a large file (e.g., 100MB), and the request is failing. What settings would you check?

Answer:

Kestrel limit:

- `MaxRequestBodySize`

IIS / Reverse Proxy:

- Request limits in `web.config` or Nginx configuration

Aaqib Khan Niazi

ASP.NET Core level:

- Use `[RequestSizeLimit(100_000_000)]` attribute

Best practice:

- Use **streaming upload** instead of buffered upload
- **Validate** file size and type **before processing**

Q: The interviewer asked you: What is the difference between a DTO and a ViewModel? How are they different from an Entity?

Answer:

- **Entity:** Domain object that maps directly to the database.
- **DTO (Data Transfer Object):** Used for data transfer — typically for API input/output.
- **ViewModel:** Frontend-specific model — includes form fields, validation logic, etc.

Q: Your API has been running continuously for 2 days and now it has slowed down. How will you check if there is a memory leak?

Answer:
Steps:

1. Use profiling tools like **dotMemory or PerfView**.
2. Check for:
    a. Static objects holding large amounts of data.
    b. Events that are not unsubscribed.
    c. Large lists or caches not being cleared.
3. In ASP.NET Core:
    a. Use IHostedService or singletons carefully, as they might retain data.
    b. Implement IDisposable properly to release resources.

Q: The JWT token does not contain roles but has claims (e.g., "Department": "HR"). How will you protect an API meant only for HR?

Answer:
I create a custom policy:

Aaqib Khan Niazi

```
services.AddAuthorization(options =>

{

    options.AddPolicy("HROnly", policy =>

        policy.RequireClaim("Department", "HR"));

});
```

Then controller :
[Authorize(Policy = "HROnly")]

Q: Your Web API is being used by a mobile app and it's showing slow performance. What will you optimize?

Answer:

1. Payload optimization: Return only the required fields
   - Use compression (GZIP)
   - Implement response caching
2. Implement network retry and resilience logic on the client side
3. Use authentication token refresh mechanism
4. Use pagination and filters to avoid over-fetching

Aaqib Khan Niazi